Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work work #3.1

Signal Acquisition. Multi-sensor Binary alerting.

During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur Cravtov
std. gr. FAF-231

Verified by:
Alexei Martîniuc
asist. univ

Chișinău, 2026

Contents

1 Objective and Technical Analysis **2**
1.1 Sensor Interfaces . . . 2
1.1.1 Digital Sensor Interface (Distance) . . . 2
1.1.2 Analog Sensor Interface (Temperature) . . . 2
1.2 Binary Signal Conditioning . . . 3
1.2.1 Threshold Detection with Hysteresis . . . 3
1.2.2 Software Debouncing via Counter Method . . . 4

2 System Design **4**
2.1 System Architecture . . . 4
2.2 Circuit Diagram . . . 5
2.2.1 HC-SR04 Ultrasonic Distance Sensor Connections . . . 5
2.2.2 NTC Thermistor Temperature Sensor Connections . . . 6
2.2.3 Circuit Connection Summary . . . 7
2.2.4 Practical Considerations . . . 7
2.3 Flowchart of the Program . . . 8

3 Implementation and Results **8**
3.1 Software Architecture . . . 8
3.2 Layered Design . . . 8
3.2.1 Distance Sensor Driver . . . 8
3.2.2 Temperature Sensor Driver . . . 9
3.3 Results . . . 10

4 Conclusion **11**

References **11**

A Source Code **12**

1 Objective and Technical Analysis

The objective of this laboratory work is to study signal acquisition and conditioning techniques for embedded systems by implementing a real-time multi-sensor monitoring system (distance and temperature) with threshold-based binary alerting. The system uses an HC-SR04 ultrasonic distance sensor and an NTC thermistor connected to an ATmega2560 microcontroller running FreeRTOS, implementing periodic sensor acquisition, binary signal conditioning through hysteresis and debouncing, and structured reporting via UART.

Through this work, the following learning outcomes are pursued:

- Understanding sensor acquisition principles for both digital (pulse-based) and analog sensors.
- Implementing the signal conditioning pipeline, including raw data transformation (thermistor resistance to Celsius).
- Implementing threshold-based binary signal detection with hysteresis to prevent oscillation near boundary values.
- Applying software debouncing techniques (counter-based) to validate the persistence of state transitions.
- Designing a modular multitask architecture for sensor acquisition, conditioning, and reporting using FreeRTOS.

1.1 Sensor Interfaces

1.1.1 Digital Sensor Interface (Distance)

Digital interface sensors provide data through pulse-based or protocol-based communication [4]. The HC-SR04 ultrasonic distance sensor uses a simple pulse-based interface: a trigger pulse is sent on the TRIG pin, and the sensor responds with a pulse on the ECHO pin whose width is proportional to the measured distance. This is a time-of-flight digital measurement read using the `pulseIn()` function. The sensor emits ultrasonic waves at 40kHz frequency, which reflect off objects and return to the receiver [5].

1.1.2 Analog Sensor Interface (Temperature)

Analog sensors provide a continuous voltage signal proportional to the measured physical quantity [4]. The NTC thermistor is part of a voltage divider circuit. The microcontroller uses its internal 10-bit Analog-to-Digital Converter (ADC) to convert the voltage at the divider's output into a digital value (0–1023). The ADC on the

ATmega2560 uses a successive approximation method with a typical conversion time of 13µs at 16MHz clock [6].

The transformation from ADC value to Celsius follows several steps:

1. **Normalization**: Mapping the 0–1023 range to 0.0–1.0.
2. **Resistance Calculation**: Based on the voltage divider formula (with a 10k fixed resistor).
3. **Temperature Calculation**: Using the B-parameter equation:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{B} \ln\left(\frac{R}{R_0}\right) \tag{1}$$

where $T_0 = 298.15K$, $R_0 = 10k\Omega$, and $B = 3950$.

1.2 Binary Signal Conditioning

Binary signal conditioning transforms a continuous sensor value into a binary decision: alert or normal.

1.2.1 Threshold Detection with Hysteresis

To prevent "chattering" near the threshold, *hysteresis* introduces two separate thresholds: a high threshold T_H and a low threshold T_L. The alert is triggered only when the value exceeds T_H, and cleared only when it drops below T_L [7]. This technique, borrowed from control systems and Schmitt trigger design, creates a "dead band" where small fluctuations do not cause state changes [8]. The concept is illustrated in Figure 1.

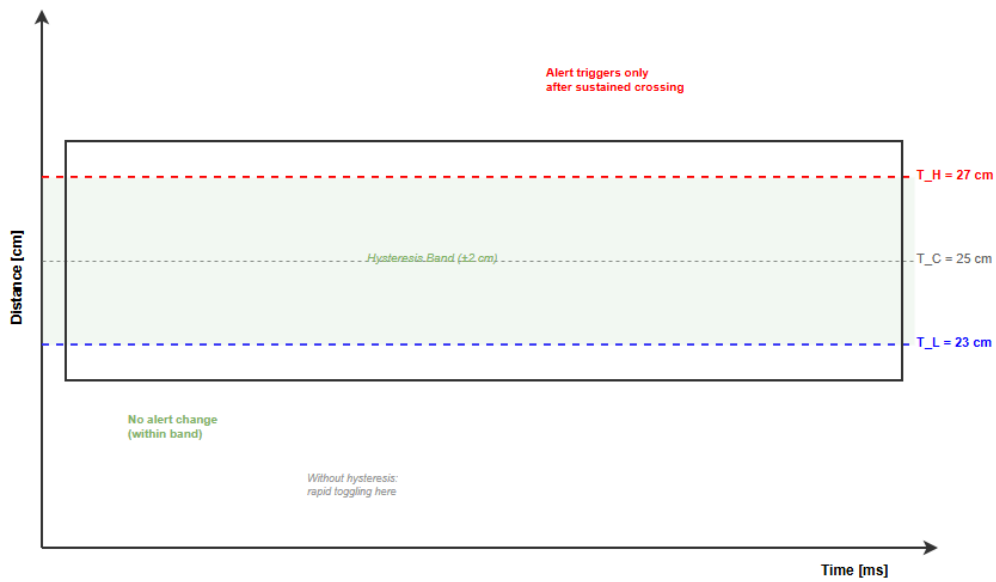

Figure 1: Threshold detection with hysteresis: two thresholds prevent oscillation

1.2.2 Software Debouncing via Counter Method

Software debouncing requires multiple consecutive readings in the same state before confirming a transition [9]. A counter increments each time the condition is met and resets if it is not. Only when the counter reaches `debounceRequired` is the transition confirmed. This counter-based approach is more robust than simple time delays, as it adapts to varying noise levels and requires genuine state persistence [10].

2 System Design

2.1 System Architecture

Figure 2 illustrates the overall architecture. The core components include:

- **Microcontroller (ATmega2560)**: Running FreeRTOS, managing concurrent tasks for acquisition, threshold detection, and reporting.
- **HC-SR04 Ultrasonic Sensor**: Connected via D3 (TRIG) and D2 (ECHO).
- **NTC Thermistor**: Connected to analog pin A0 in a voltage divider.
- **UART Interface**: Used for structured reporting at 9600 baud.
- **FreeRTOS Kernel**: Manages task scheduling and inter-task communication via Queues and Mutexes.

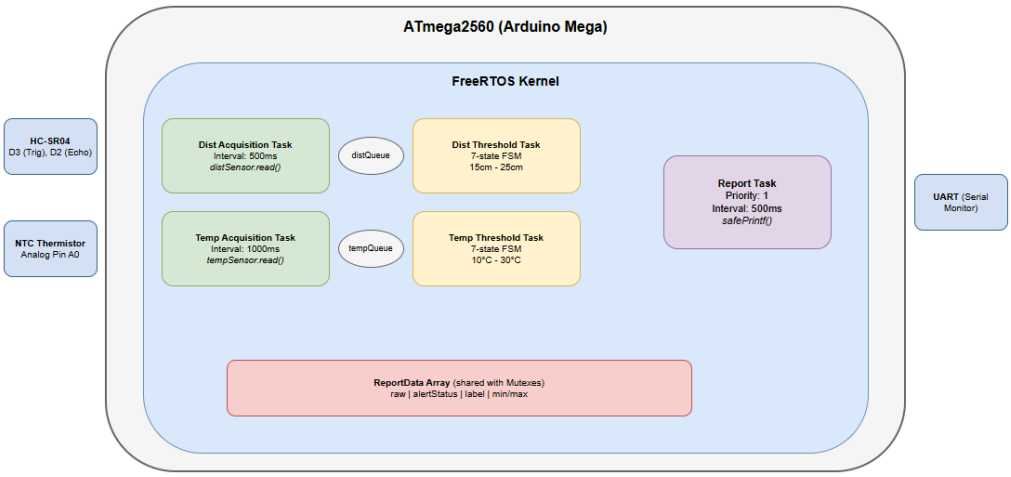

Figure 2: System Architecture Diagram

2.2 Circuit Diagram

Figure 3 shows the hardware connections between the ATmega2560 microcontroller and the sensors. The circuit consists of two main sensor interfaces connected to the microcontroller, each with distinct signal characteristics and interfacing requirements.

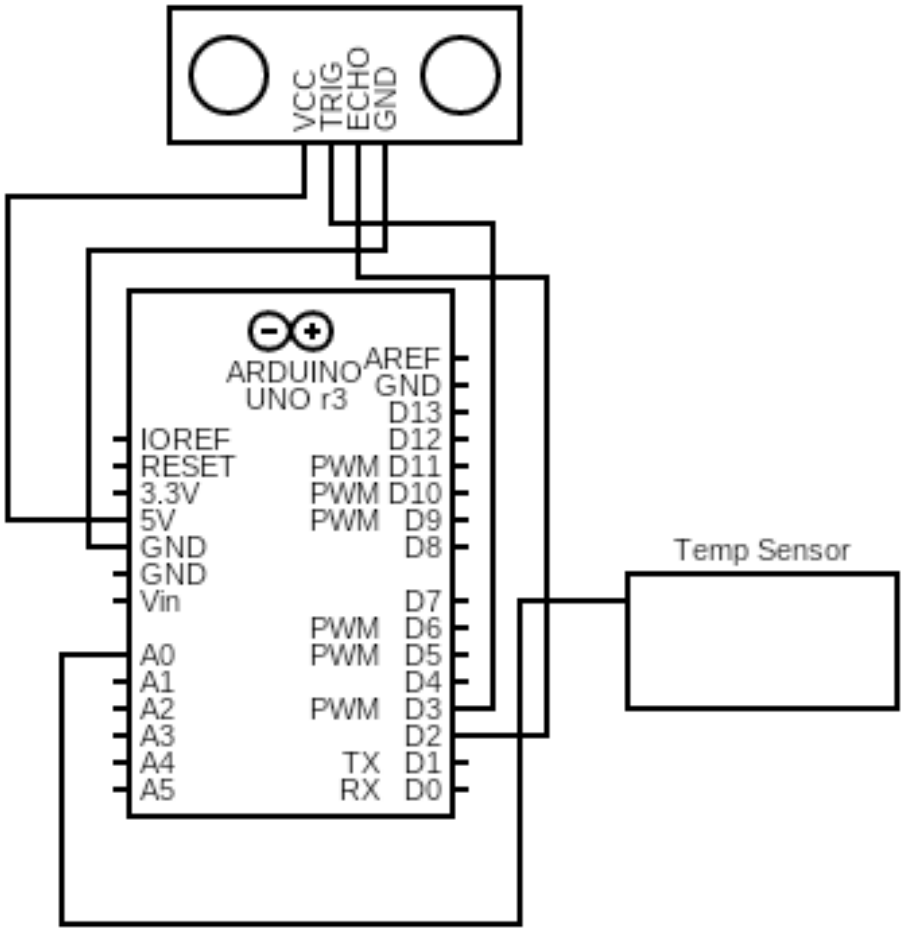

Figure 3: Circuit Diagram: Sensor Connections

2.2.1 HC-SR04 Ultrasonic Distance Sensor Connections

The HC-SR04 is a widely-used ultrasonic ranging module that provides 2cm to 400cm of non-contact measurement functionality with a ranging accuracy that can reach up to 3mm [2]. The sensor has four pins that connect directly to the microcontroller:

- **VCC**: Connected to the 5V power supply rail. The sensor requires a stable 5V supply for proper operation. The current consumption is approximately 15mA during measurement, making it suitable for battery-powered applications with appropriate power management.

- **GND**: Connected to the ground rail, providing the common reference potential for all signals. A common ground is essential for accurate timing measurements.
- **TRIG (Trigger)**: Connected to digital pin D3 of the ATmega2560. This input pin receives a 10µs HIGH pulse from the microcontroller to initiate a measurement cycle. The sensor's internal circuitry detects the rising edge and begins the ultrasonic burst.
- **ECHO**: Connected to digital pin D2 of the ATmega2560. This output pin provides a pulse whose duration is proportional to the measured distance. The relationship is approximately 58µs per centimeter, derived from the speed of sound (343 m/s at 20°C).

The ultrasonic sensor operates on the time-of-flight principle. When triggered, it emits eight 40kHz ultrasonic pulses and simultaneously sets the ECHO pin HIGH. When the reflected sound waves are detected, the ECHO pin goes LOW. The distance is calculated as:

$$d = \frac{t_{echo} \times v_{sound}}{2} \tag{2}$$

where t_{echo} is the pulse duration and $v_{sound} \approx 343$ m/s. The division by 2 accounts for the round-trip of the sound wave.

2.2.2 NTC Thermistor Temperature Sensor Connections

The NTC (Negative Temperature Coefficient) thermistor is a semiconductor device whose resistance decreases predictably with increasing temperature [3]. It is connected in a voltage divider configuration:

- **Voltage Divider Circuit**: The NTC thermistor (nominal resistance 10kΩ at 25°C) is connected in series with a fixed 10kΩ resistor between VCC (5V) and GND. The choice of the fixed resistor value (equal to the thermistor's nominal resistance) provides maximum sensitivity near 25°C.
- **Output Signal**: The junction between the thermistor and the fixed resistor is connected to analog input pin A0 of the ATmega2560. The output voltage varies between approximately 0.5V (hot) and 4.5V (cold) within the operating range.
- **ADC Reference**: The internal 10-bit ADC uses AVCC (5V) as the reference voltage, providing a resolution of approximately 4.9mV per step. For temperature measurements, this translates to roughly 0.25°C resolution near room temperature.

The voltage divider arrangement allows the microcontroller to measure the thermistor's resistance indirectly through voltage measurement. The relationship between the

output voltage V_{out} and the thermistor resistance R_{NTC} is given by:

$$V_{out} = V_{CC} \cdot \frac{R_{fixed}}{R_{NTC} + R_{fixed}} \tag{3}$$

As temperature increases, the NTC resistance decreases (hence "negative temperature coefficient"), causing the output voltage at A0 to rise. This behavior is characteristic of NTC thermistors and is exploited for temperature measurement.

2.2.3 Circuit Connection Summary

Table 1 summarizes the complete pin assignments for the sensor circuit. All connections are direct, without intermediate components such as level shifters, as all components operate at 5V logic levels compatible with the ATmega2560.

Table 1: Microcontroller Pin Assignments

Component	**Pin**	**Function**
HC-SR04 VCC	5V rail	Power supply
HC-SR04 GND	GND rail	Ground reference
HC-SR04 TRIG	D3	Trigger input (digital output)
HC-SR04 ECHO	D2	Echo output (digital input)
NTC Thermistor (top)	5V rail	Connected to VCC
NTC Thermistor (bottom)	A0 via junction	ADC input
10kΩ Fixed Resistor (top)	A0 via junction	ADC input
10kΩ Fixed Resistor (bottom)	GND rail	Ground reference

2.2.4 Practical Considerations

Several practical factors were considered in the circuit design:

- **Debouncing**: While the HC-SR04 provides relatively clean signals, the ECHO pulse timing can be affected by acoustic reflections in enclosed spaces. The software implements a debounce counter to filter spurious readings.
- **ADC Noise**: The thermistor circuit is susceptible to electrical noise, especially in environments with switching loads. A small capacitor (0.1µF) can be added between A0 and GND to filter high-frequency noise.
- **Self-heating**: Passing current through the thermistor causes self-heating, which can introduce measurement errors. The voltage divider current is approximately 250µA at 25°C, which minimizes self-heating effects.

2.3 Flowchart of the Program

The system uses separate tasks for each sensor's acquisition and threshold monitoring, allowing different sampling rates and threshold parameters.

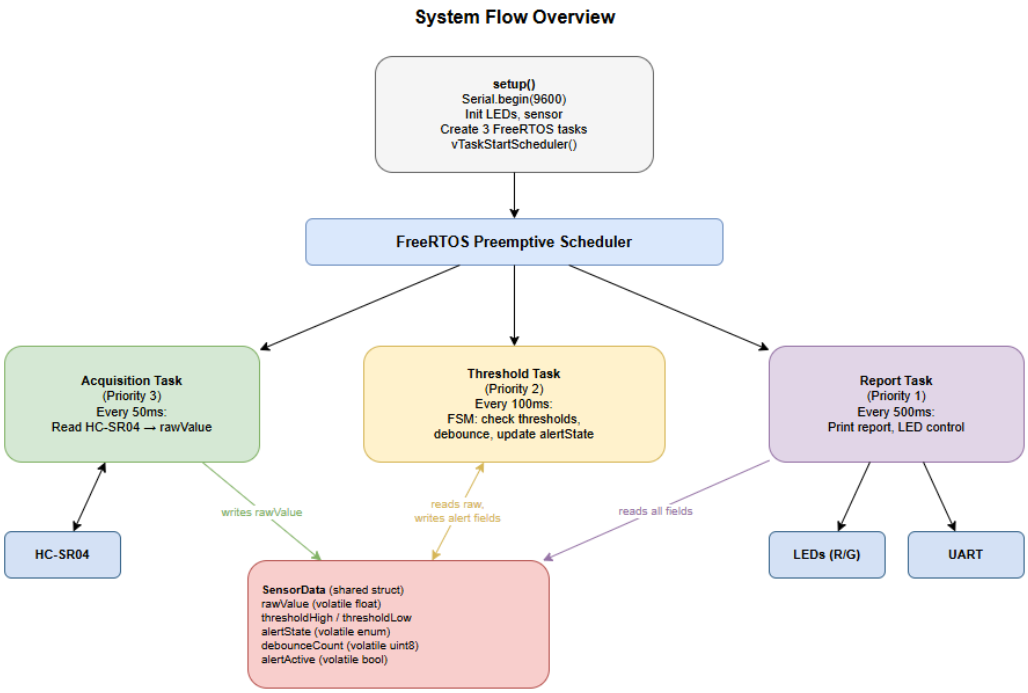

Figure 4: General System Flowchart: Task Interaction

3 Implementation and Results

3.1 Software Architecture

The implementation leverages FreeRTOS tasks and C++ classes for modularity.

- **SensorAquisitionTask**: Periodically reads the sensor and sends the value to a queue.
- **ThreshHoldTask**: Receives values from the queue and applies the 7-state FSM.
- **ReportTask**: Aggregates data from shared `ReportData` structures and prints to Serial.

3.2 Layered Design

3.2.1 Distance Sensor Driver

Figure 5 shows the layer decomposition for the pulse-based digital sensor.

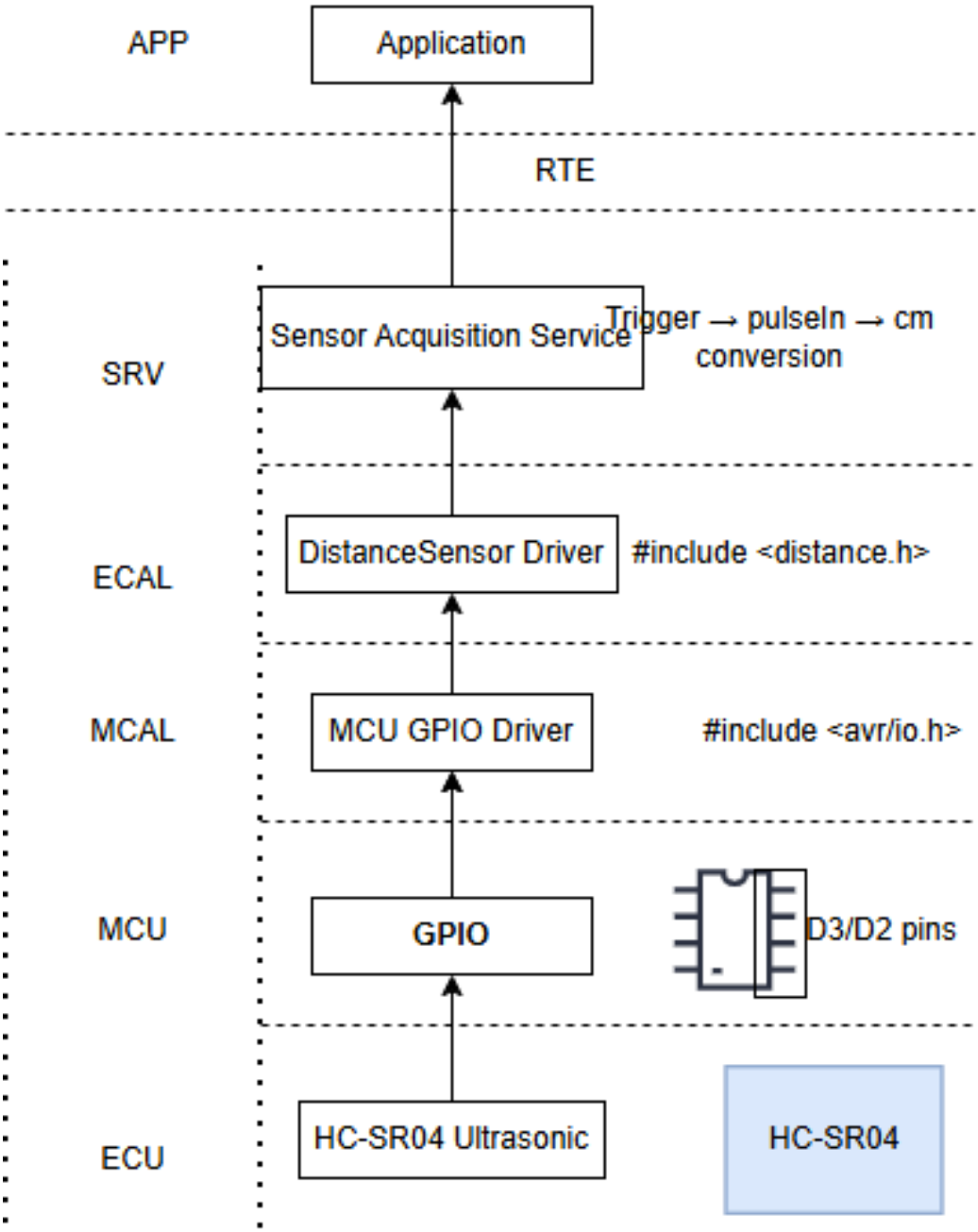

Figure 5: Distance Sensor layering

3.2.2 Temperature Sensor Driver

Figure 6 shows the layer decomposition for the analog sensor. The ECAL layer here includes the mathematical transformation from ADC counts to Celsius.

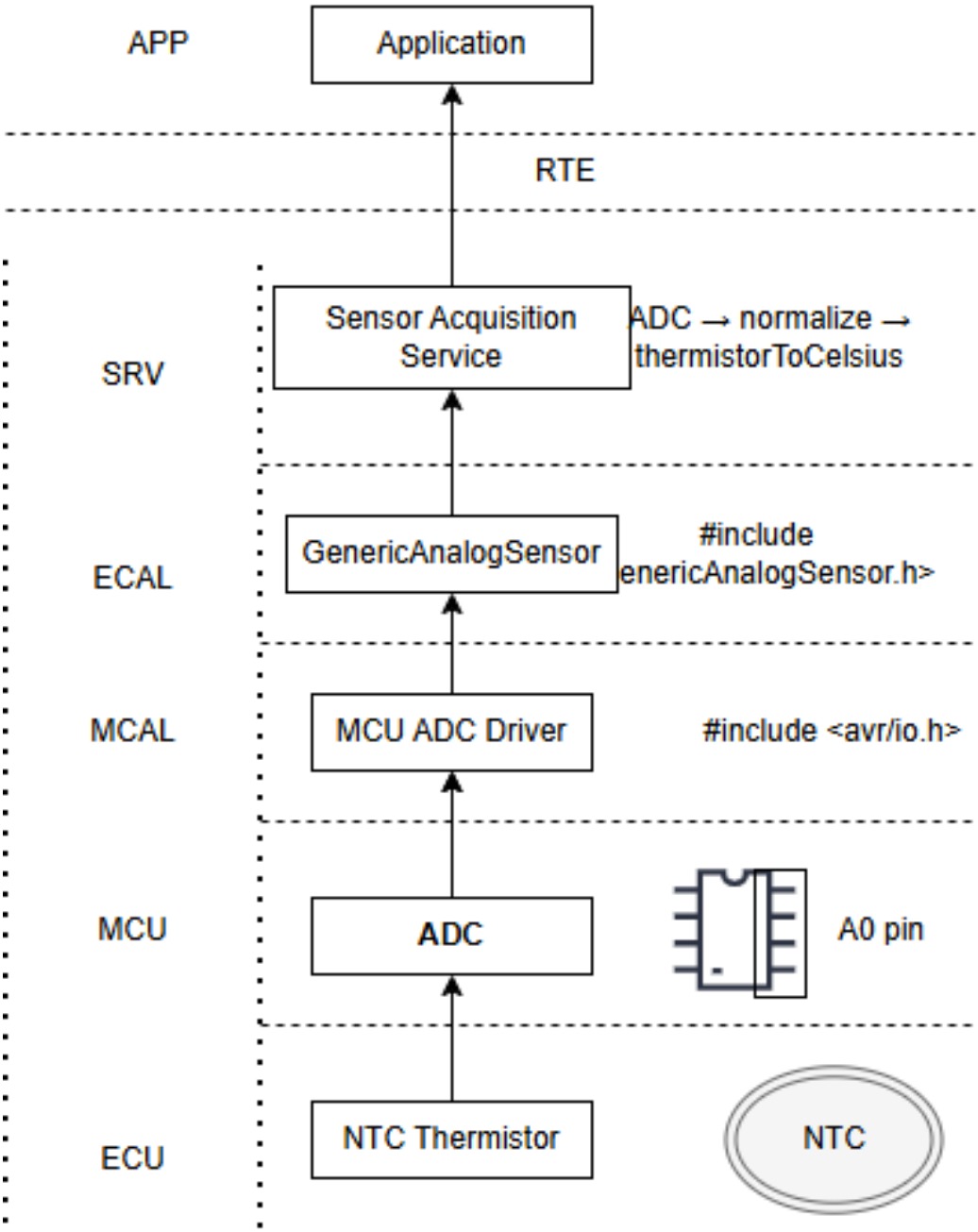

Figure 6: Temperature Sensor layering

3.3 Results

The system provides real-time monitoring of both sensors. For the distance sensor, the threshold is set at $15cm \ldots 25cm$. For the temperature sensor, the range is $10°C \ldots 30°C$.

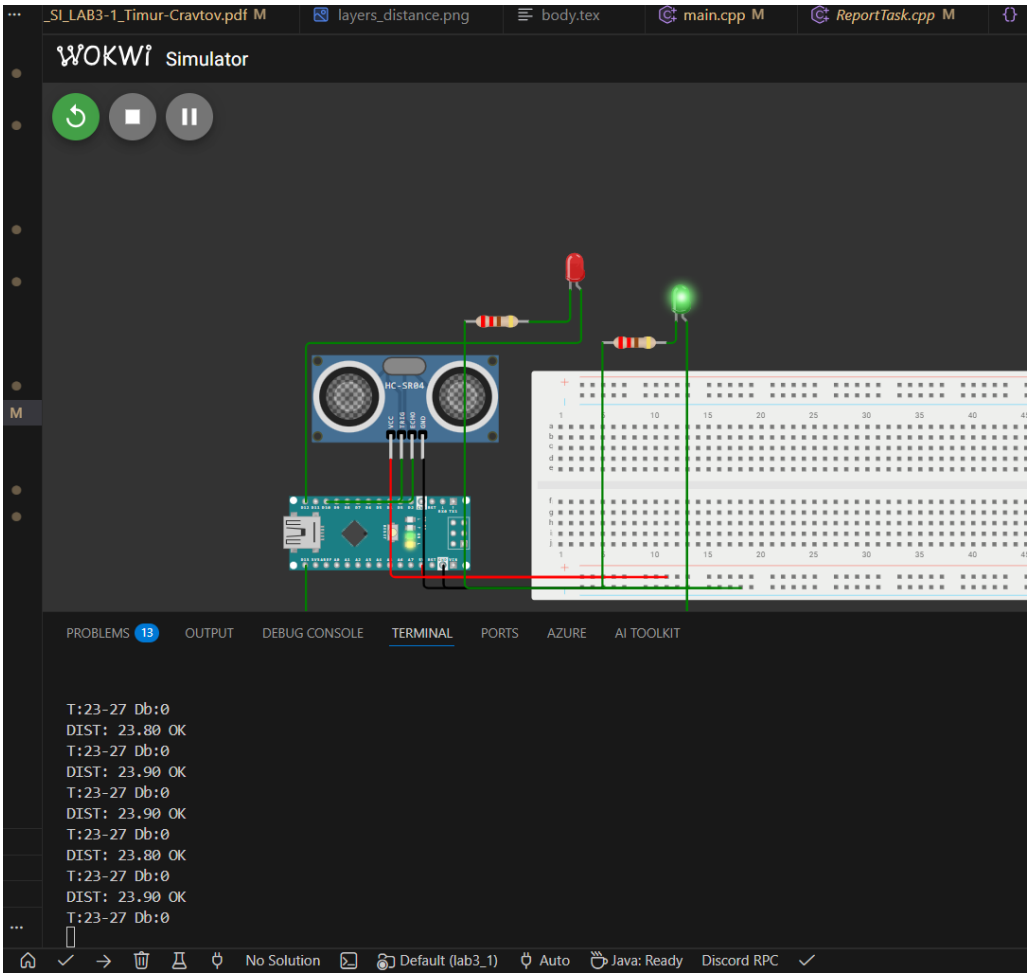

Figure 7: Simulation Result: Dual Sensor Normal Operation

4 Conclusion

This laboratory work demonstrated a robust multi-sensor acquisition and binary conditioning pipeline. By using FreeRTOS, the system maintains independent timing for distance measurements (500ms) and temperature sampling (1000ms), while the reporting task provides a unified view every 500ms. The implementation of hysteresis and debouncing ensures that alerts are only triggered by legitimate environmental changes, making the system suitable for real-world IoT applications.

References

[1] FreeRTOS Real Time Kernel, `https://www.freertos.org/`

[2] HC-SR04 Ultrasonic Ranging Module Datasheet, `https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf`

[3] NTC Thermistor Principles and Applications, Vishay, https://www.vishay.com/docs/29053/ntccomp.pdf

[4] Fraden, J., "Handbook of Modern Sensors: Physics, Designs, and Applications", 5th ed., Springer, 2016.

[5] Raj, B. et al., "Ultrasonic Testing: Principles and Applications", Springer, 2022.

[6] ATmega2560 Datasheet, Microchip Technology, https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-2549-8-bit-AVR-Microcontroller-ATmega640-1280-1281-2560-2561_datasheet.pdf

[7] Astrom, K.J., Murray, R.M., "Feedback Systems: An Introduction for Scientists and Engineers", Princeton University Press, 2020.

[8] Schmitt, O.H., "A Thermionic Trigger", Journal of Scientific Instruments, Vol. 15, No. 1, 1938.

[9] Ganssle, J., "A Guide to Debouncing", The Ganssle Group, 2008.

[10] Barr, M., Massa, A., "Programming Embedded Systems in C and C++", O'Reilly Media, 2006.

A Source Code

Full source code for Lab 3.1 is available in the repository.

main.cpp

```
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <serialio/serialio.h>
#include <math.h>

#include "SafePrintf.h"
#include "GenericAnalogSensor.h"
#include "SonicDistanceSensor.h"
#include "SensorAquisitionTask.h"
#include "ThreshHoldTask.h"
#include "ReportTask.h"

// —— Transformations ——
float thermistorToCelsius(float normalized) {
```

```
    if (normalized <= 0.001f || normalized >= 0.999f) return 0.0f;
    const float B  = 3950.0f;
    const float R0 = 10000.0f;
    const float T0 = 298.15f;
    float resistance = 10000.0f * (normalized / (1.0f - normalized));
    float tempK = 1.0f / ((1.0f / T0) + (1.0f / B) * log(resistance / R0));
    return tempK - 273.15f;
}

// —— Static Allocations ——
GenericAnalogSensor tempSensor(A0, thermistorToCelsius);
SonicDistanceSensor distSensor(3, 2);

SensorAquisitionTask tempTaskObj("Temp (C)", &tempSensor, 1000);
SensorAquisitionTask distTaskObj("Dist (cm)", &distSensor, 500);

void distAlertCb() { safePrintf("ALERT: Distance out of range!\n"); }
void tempAlertCb() { safePrintf("ALERT: Temperature out of range!\n"); }

ThreshHoldTask distAlertTaskObj(15.0f, 25.0f, 100, 3, distAlertCb);
ThreshHoldTask tempAlertTaskObj(10.0f, 30.0f, 100, 3, tempAlertCb);

ReportData reportDataArray[2] = {
    {0, "Temp (C)",  0.0f, NULL},
    {0, "Dist (cm)", 0.0f, NULL}
};
ReportTask reportTaskObj(reportDataArray, 2, 500);

QueueHandle_t distQueue;
QueueHandle_t tempQueue;
```

```

void setup() {
    Serial.begin(9600);
    redirectSerialToStdio();

    safePrintfInit();

    distQueue = xQueueCreate(10, sizeof(float));
    tempQueue = xQueueCreate(10, sizeof(float));

    reportDataArray[0].xSemaphore = xSemaphoreCreateMutex();
    reportDataArray[1].xSemaphore = xSemaphoreCreateMutex();

    tempTaskObj.setQueue(tempQueue);
    distTaskObj.setQueue(distQueue);
    tempAlertTaskObj.setQueue(tempQueue);
    distAlertTaskObj.setQueue(distQueue);

    tempTaskObj.setReportEntry(&reportDataArray[0]);
    distTaskObj.setReportEntry(&reportDataArray[1]);
    tempAlertTaskObj.setReportEntry(&reportDataArray[0]);
    distAlertTaskObj.setReportEntry(&reportDataArray[1]);

    xTaskCreate([](void* p) { static_cast<SensorAquisitionTask*>(p)->run(); }, "TempTask", 160, &tempTaskObj, 1, NULL);
    xTaskCreate([](void* p) { static_cast<SensorAquisitionTask*>(p)->run(); }, "DistTask", 160, &distTaskObj, 1, NULL);
    xTaskCreate([](void* p) { static_cast<ThreshHoldTask*>(p)->run(); }, "AlertDist", 128, &distAlertTaskObj, 1, NULL);
    xTaskCreate([](void* p) { static_cast<ThreshHoldTask*>(p)->run(); }, "AlertTemp", 128, &tempAlertTaskObj, 1, NULL);
    xTaskCreate([](void* p) { static_cast<ReportTask*>(p)->run(); }, "ReportTask", 256, &reportTaskObj, 1, NULL);
```

```

    vTaskStartScheduler();
}

void loop() {}
```

Listing 1: main.cpp (lab3_1)